

Project name

AI Career Companion

AGILE PROJECT DOCUMENTATION

Project title

AI Career Companion Agent for Internship Matching and Interview Preparation

Prepared by

Bhanushree K

Project Duration

July 22 – September 22

Mentor

Vijay Parmar

Index

Index.....	2
1. Project Overview	4
1.1 Project Name.....	4
1.2 What is the Project?	4
1.3 Problem Statement	4
1.4 Target Users.....	4
2. Project Objective.....	5
Objectives	5
3. Technology Stack.....	6
4. Project Requirements	7
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	7
5. System Architecture / Workflow	8
5.1 Resume Parsing Flow	8
5.2 Internship Matching Flow	8
5.3 Database Schema	9
6. Product Backlog.....	10
7. Sprint Planning	11
Sprint 1 — Goal: Project foundation and authentication [1 st and 2 nd week].....	11
Sprint 2 — Goal: Resume intelligence and internship discovery [3 rd and 4 th week].....	11
Sprint 3 — Goal: Semantic matching and career assistance [5 th and 6 th week]	11
Sprint 4 — Goal: AI assistants, integration and final validation [7 th and 8 th week]	11
8. Sprint Tasks & Status	12
9. Implementation / Key Features.....	13
9.1 Account Creation and Login.....	13
9.2 Dashboard.....	14
9.3 Resume Parsing	14
9.4 Internships	15
9.5 My Matches	16
9.6 Cover Letter	16
9.7 AI Product Assistant.....	17
9.8 Interview Preparation Agent.....	18
9.9 Profile	18

9.10 Additional Features	19
Parsed Resumes	19
Applied Internships	20
Light / Dark Theme	20
10. Testing	21
Testing Types	21
11. Challenges & Solutions	23
12. Final Project Status	24
12.1 Completed Features	24
12.2 Final Submission Status	24
12.3 Final Outcome	24
12.4 Conclusion	24

1. Project Overview

1.1 Project Name

AI Career Companion Agent for Internship Matching and Interview Preparation

1.2 What is the Project?

AI Career Companion is a web-based career assistance application designed to help students and internship seekers move from resume analysis to internship discovery, application support, AI-assisted product guidance and interview preparation. The application accepts PDF or DOCX resumes, extracts structured career information using a hybrid Regex + Groq LLM pipeline, and uses the resulting profile as the primary input for semantic internship matching.

The system also provides an internship catalog, matched opportunities, in-app applications, tailored cover-letter generation, a manually managed Career Profile, an AI Product Assistant developed as part of the internship feature set, and a separate AI Interview Preparation Agent that was developed subsequently and uses resume context, optional uploaded documents, and session conversation history.

1.3 Problem Statement

Students and internship seekers often have to perform several separate tasks: understand the information contained in their resume, search through internship listings, judge whether an opportunity fits their skills and education, prepare for technical and HR/behavioral interviews, and write application material. The project brings these activities into one application and uses AI-assisted processing to make the workflow more personalized.

The matching component is designed to provide not only a ranked opportunity list but also match percentage, match label, matched skills, skill gaps, and an AI-generated summary of the overall result set.

1.4 Target Users

- Students searching for internships.
- Candidates who want resume-based internship recommendations.
- Internship seekers preparing for technical and HR/behavioral interviews.
- Users who want application support such as tailored cover letters.

2. Project Objective

The main objective is to provide an integrated AI-assisted career companion that uses a candidate's resume and related context to support internship discovery, application assistance, product guidance and interview preparation.

Objectives

- Allow users to register, authenticate securely, and maintain user-specific application data.
- Parse PDF and DOCX resumes into structured career information using a hybrid Regex + LLM approach.
- Use sentence-transformer embeddings and FAISS to retrieve semantically relevant internship opportunities.
- Re-rank retrieved internships using skills, semantic similarity, education, and location signals.
- Show match explanations through matched skills, skill gaps, match percentage, and match label.
- Provide a separate Product Assistant that uses a product-knowledge document and FAISS retrieval before generating an answer with Groq.
- Provide an AI Interview Preparation Agent that can use parsed resume context, uploaded PDF/DOCX documents, and conversation history.
- Generate internship-specific cover letters using only the supplied candidate and internship information.
- Allow users to browse internships, apply within the application, and view their application history.
- Provide a manually maintained Career Profile separate from the parsed-resume data used for matching.

Scope note: The project contains both internship-matching retrieval and a separate product-knowledge RAG assistant. The Interview Preparation Agent is a separate service and should not be described as the same component as the Product Assistant.

3. Technology Stack

Category	Technology / Usage
Frontend	React 18.3.1, Vite, JavaScript, React Router, CSS, React Markdown, react-easy-crop
Backend	Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy
Database	PostgreSQL
Authentication	JWT with python-jose; password hashing with Passlib/bcrypt
AI / LLM	Groq API, LangChain, LangChain-Groq, prompt-based LLM extraction and generation
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Vector Search	FAISS (faiss-cpu)
Document Parsing	PyMuPDF for PDF; python-docx for DOCX
Matching	FAISS semantic retrieval + skills, education and location re-ranking
AI Product Assistant	Product-knowledge document + FAISS retrieval + Groq
Data / Persistence	PostgreSQL JSONB for parsed/profile data and application/session records
Version Control	Git / GitHub

4. Project Requirements

4.1 Functional Requirements

- User registration and login with JWT-based authentication.
- Password reset and password change flows.
- Protected application routes and user-specific data access.
- Upload PDF or DOCX resumes with file-type and size validation.
- Extract resume text and convert it into structured profile data.
- Store uploaded resumes and parsed-resume records for later reuse and re-parsing.
- Browse, search, and filter the internship catalog.
- Generate a selected number of top internship matches using FAISS-based retrieval and re-ranking.
- Display match percentage, match label, matched skills, and skill gaps.
- Apply to internships inside the application and view applied internships.
- Generate and save internship-specific cover letters.
- Maintain a separate user-managed Career Profile.
- Use the AI Product Assistant with product-knowledge retrieval and chat-session history.
- Use the Interview Preparation Agent for resume-based, document-based, technical, HR, and company-specific preparation.

4.2 Non-Functional Requirements

- Security: authenticated routes and user ownership checks for stored data.
- Reliability: validation and error handling for unsupported files and failed AI/API operations.
- Usability: clear workflows for resume parsing, matching, application, and interview preparation.
- Maintainability: separate routers and services for authentication, resume parsing, matching, chat, interview preparation, and career features.
- Readability: AI responses are presented through the frontend as readable Markdown where supported.
- Performance consideration: embedding models and FAISS indexes are cached per backend process to avoid repeated loading.

5. System Architecture / Workflow

The application follows a React/Vite frontend and FastAPI backend architecture. PostgreSQL stores persistent application data, while FAISS provides local vector retrieval. Groq is used for LLM-powered extraction and generation, and the local sentence-transformer model is used for semantic embeddings.

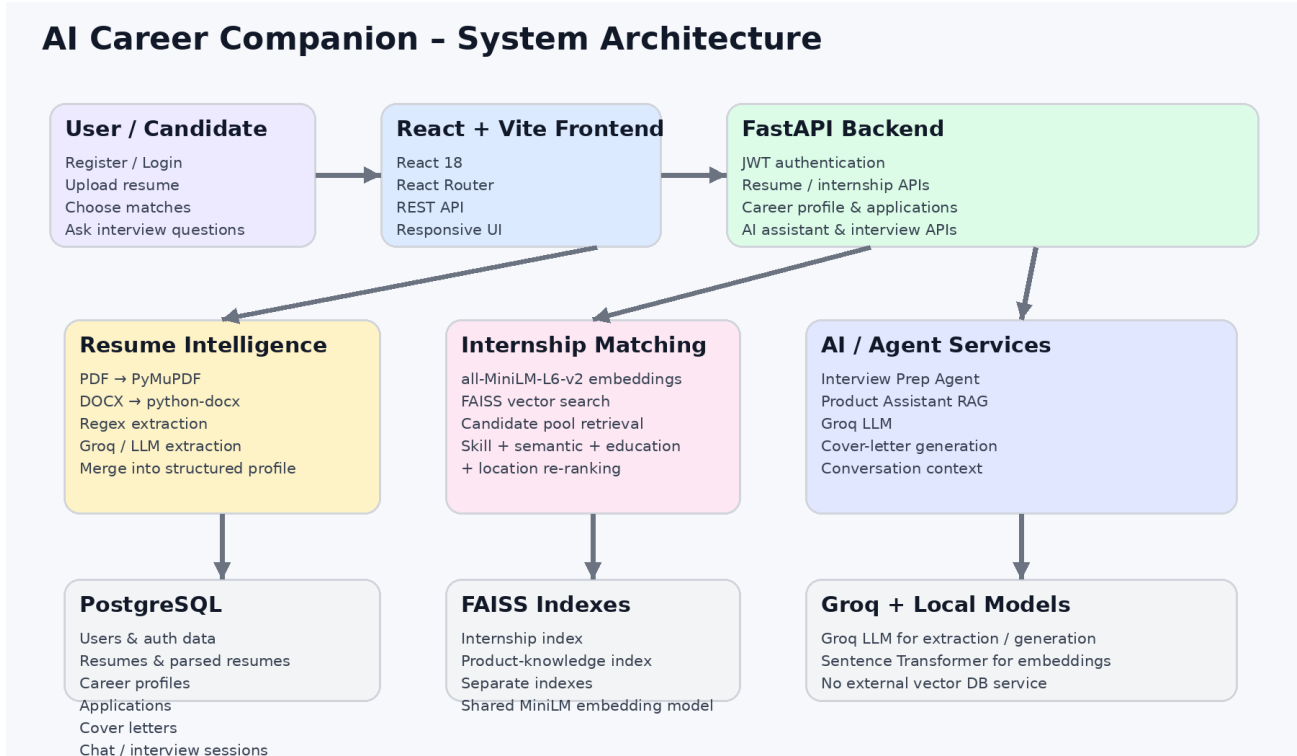


Figure 1. System architecture.

5.1 Resume Parsing Flow

1. User uploads a PDF or DOCX through the React frontend.
2. FastAPI validates the extension and upload size and stores the original file.
3. PyMuPDF or python-docx extracts the document text.
4. Regex extraction identifies deterministic contact information and other pattern-based fields.
5. Groq/LLM extraction identifies semantic fields such as summary, skills, education, experience, projects, certifications, achievements, and related information.
6. The merge service combines the two outputs, using Regex as authoritative for precise contact fields and LLM output for contextual/semantic fields.
7. The final structured profile, together with the source resume, is stored in PostgreSQL.

5.2 Internship Matching Flow

8. The latest successfully parsed resume is converted into a clean query containing skills, education, summary, and supporting project/experience text.
9. The same all-MiniLM-L6-v2 embedding model is used for the resume query and internship postings.
10. FAISS retrieves a larger candidate pool of semantically similar internship postings.
11. The candidates are re-ranked using weighted skill, semantic, education, and location scores.

12. The top-k results are returned with match percentage, label, matched skills, missing skills, and an optional Groq summary.

5.3 Database Schema

The application uses PostgreSQL with SQLAlchemy for persistent application data. The schema stores user accounts, uploaded resumes, parsed resume data, the manually maintained Career Profile, cover letters, internship applications, authentication support records, Product Assistant conversations, and Interview Preparation sessions. UUIDs are used as primary keys for the main entities, while JSONB is used for structured resume/profile data and optional conversational sources.

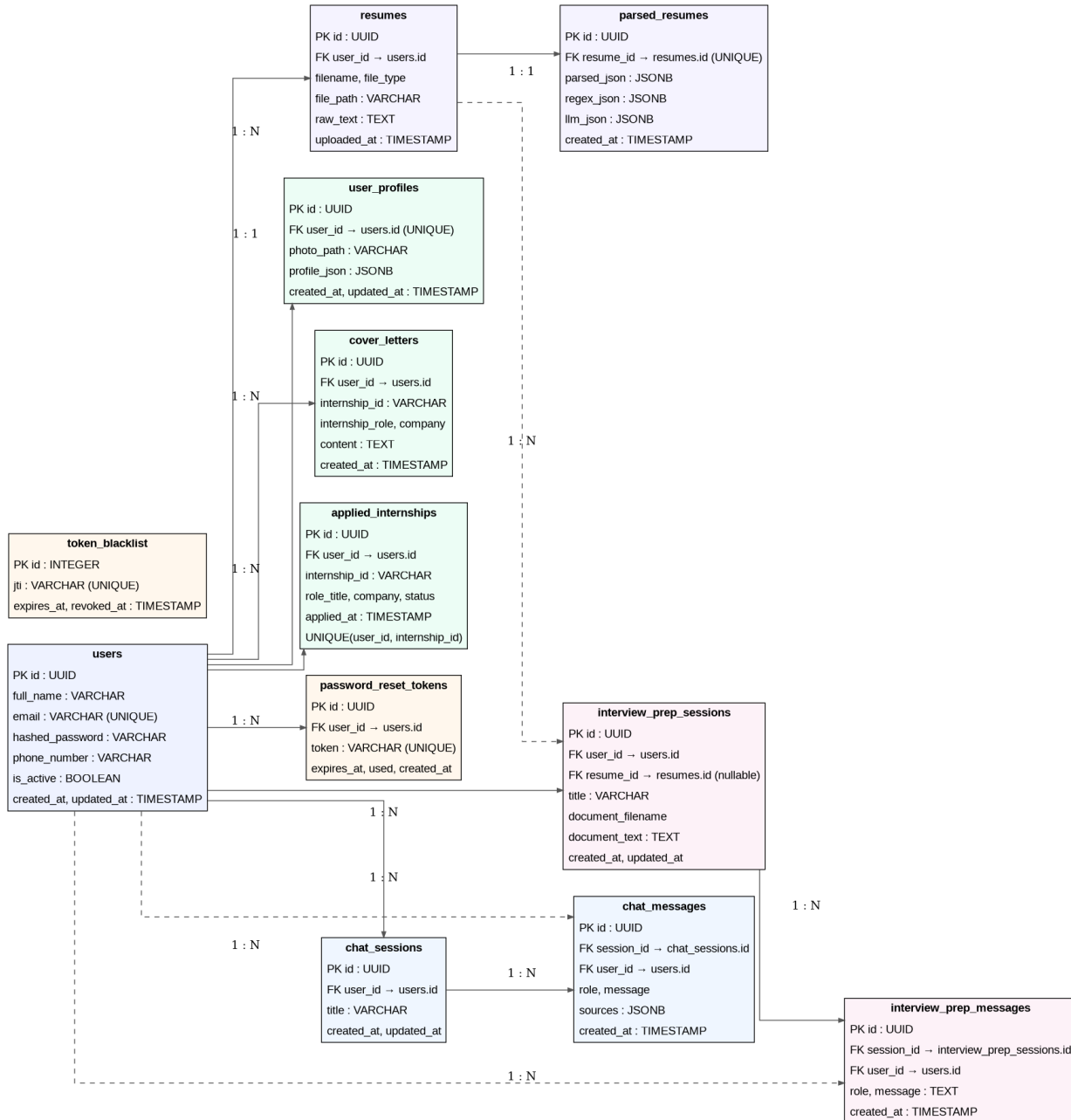


Figure 2. Database schema

6. Product Backlog

The backlog represents the major implemented feature areas found in the repository. Status is based on the presence of the corresponding working code in the final project.

ID	Feature / Task	Priority	Status
P001	Project structure and development setup	High	Done
P002	User registration, login, JWT authentication	High	Done
P003	PostgreSQL models and persistence	High	Done
P004	PDF/DOCX resume upload and extraction	High	Done
P005	Hybrid Regex + Groq LLM resume parser	High	Done
P006	Internship catalog, search and filtering	High	Done
P007	FAISS semantic internship matching and re-ranking	High	Done
P008	Career Profile management	Medium	Done
P009	Applied Internships tracking	Medium	Done
P010	AI-generated internship-specific cover letters	Medium	Done
P011	AI Product Assistant with product-knowledge RAG	High	Done
P012	AI Interview Preparation Agent	High	Done
P013	Frontend workflow, routing and theme support	Medium	Done
P014	Final validation, documentation and demo preparation	High	Done

Priority values: High, Medium, Low. Status values: To Do, In Progress, Testing, Done.

7. Sprint Planning

The project was organized into four development cycles.

Sprint 1 — Goal: Project foundation and authentication [1st and 2nd week]

- Set up frontend and backend structure.
- Configure FastAPI, React/Vite and PostgreSQL connectivity.
- Implement registration, login, JWT authentication and protected routes.
- Create core database models and persistence.

Sprint 2 — Goal: Resume intelligence and internship discovery [3rd and 4th week]

- Implement PDF/DOCX text extraction.
- Implement Regex and Groq LLM resume extraction.
- Merge extracted information into structured resume data.
- Create internship catalog, search/filtering and internship data handling.

Sprint 3 — Goal: Semantic matching and career assistance [5th and 6th week]

- Build the internship FAISS index.
- Implement all-MiniLM-L6-v2 embeddings.
- Implement candidate retrieval and skill/semantic/education/location re-ranking.
- Add Career Profile, Applied Internships and Cover Letter features.

Sprint 4 — Goal: AI assistants, integration and final validation [7th and 8th week]

- Implement Product Assistant RAG using a separate product-knowledge FAISS index.
- Implement the separate Interview Preparation Agent.
- Integrate frontend routes and shared UI.
- Validate registration/login and user-selected match count workflow.
- Prepare final documentation and demo materials.

8. Sprint Tasks & Status

Sprint	Task	Status
Sprint 1	Project setup / repository / backend and frontend foundation	Done
Sprint 1	Authentication and protected application routes	Done
Sprint 1	Database models and PostgreSQL persistence	Done
Sprint 2	PDF/DOCX parsing and hybrid resume extraction	Done
Sprint 2	Structured parsed-resume storage and history	Done
Sprint 2	Internship catalog and filtering	Done
Sprint 3	FAISS internship index and MiniLM embeddings	Done
Sprint 3	Composite internship ranking and match explanations	Done
Sprint 3	Career Profile, applications and cover letters	Done
Sprint 4	Product Assistant RAG	Done
Sprint 4	Interview Preparation Agent	Done
Sprint 4	Final integration / UI validation / documentation	Done

9. Implementation / Key Features

The following sections describe the major features implemented in the application.

9.1 Account Creation and Login

Feature: Users can create an account and then sign in manually. Registration does not automatically authenticate the new account.

How it works: The React registration form sends account details to the FastAPI registration endpoint. After successful registration, the application navigates to the Login page. Authentication is then performed through the login endpoint, which issues the JWT access token used by protected routes.

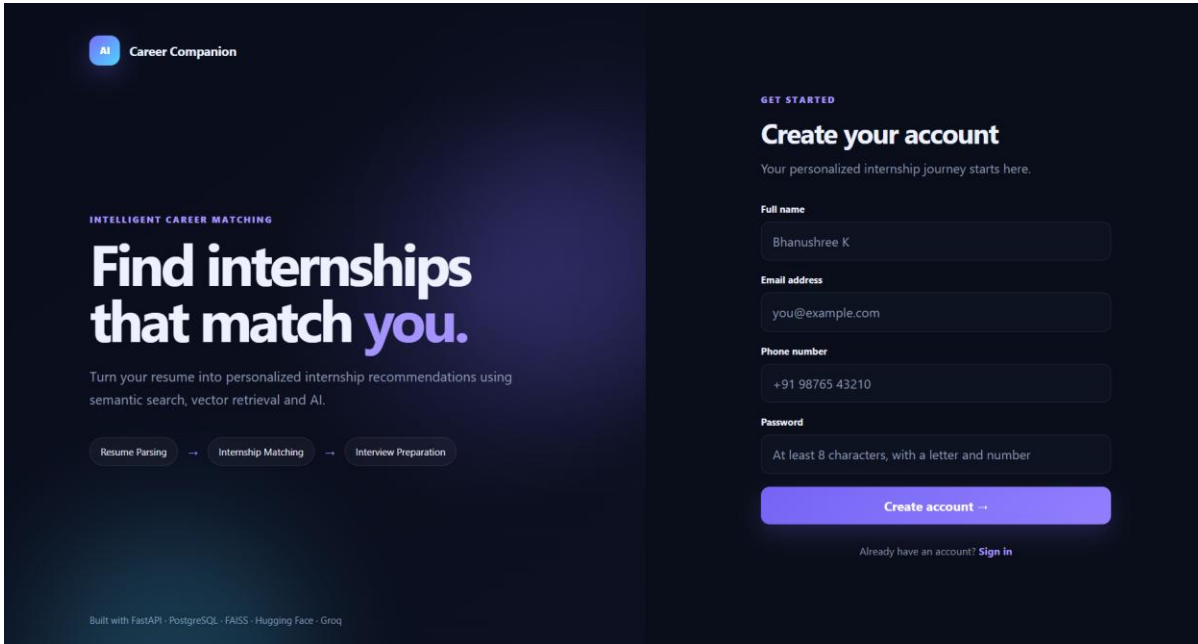


Figure 2. Create Account / Register page.

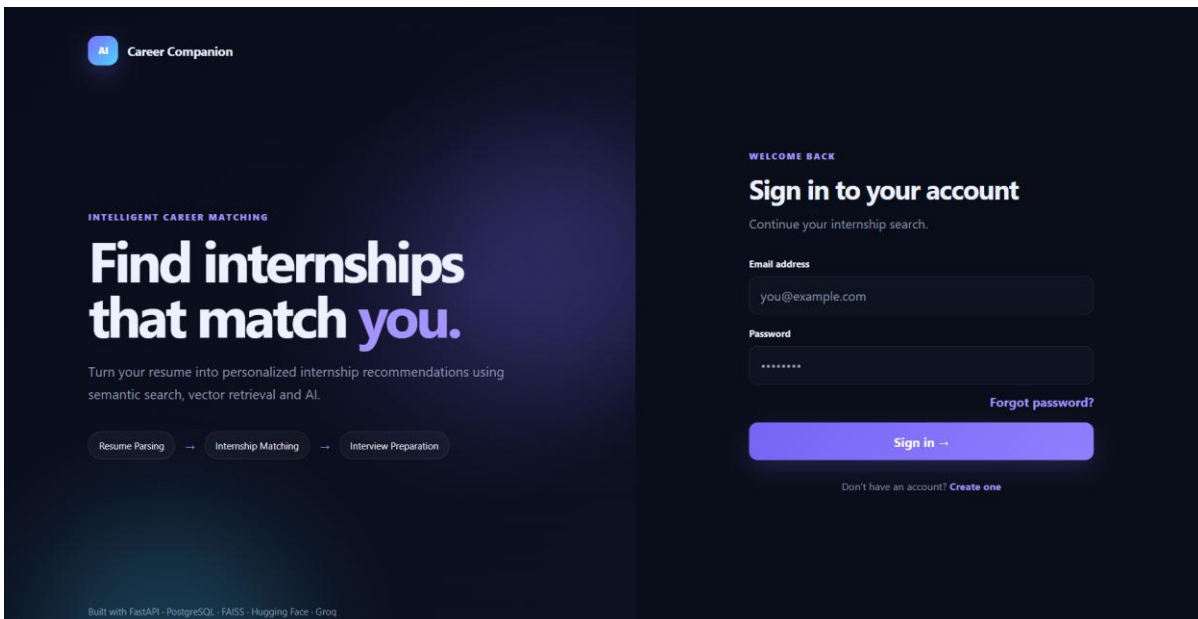


Figure 3. Login page.

9.2 Dashboard

Feature: The Dashboard is the main landing page after successful login and provides an overview of the career-assistance workflow.

How it works: The dashboard presents the resume-to-matching workflow and summarizes the technologies supporting internship matching, including FAISS, the MiniLM embedding model and Groq AI analysis. It also provides the entry point for beginning resume parsing.

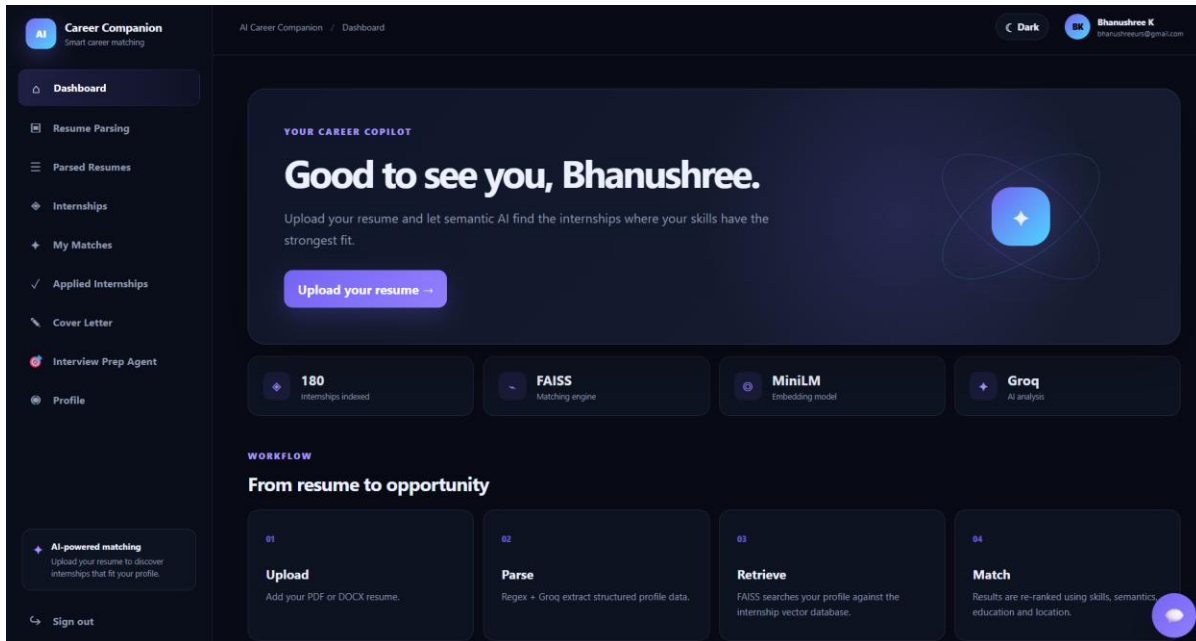


Figure 4. Application Dashboard.

9.3 Resume Parsing

Feature: Users upload a PDF or DOCX resume and receive structured career information.

How it works: The backend validates the file, extracts text with PyMuPDF or python-docx, performs Regex extraction for deterministic fields, and uses Groq/LLM extraction for contextual fields. The outputs are merged into the structured parsed-resume record stored for the user's account.

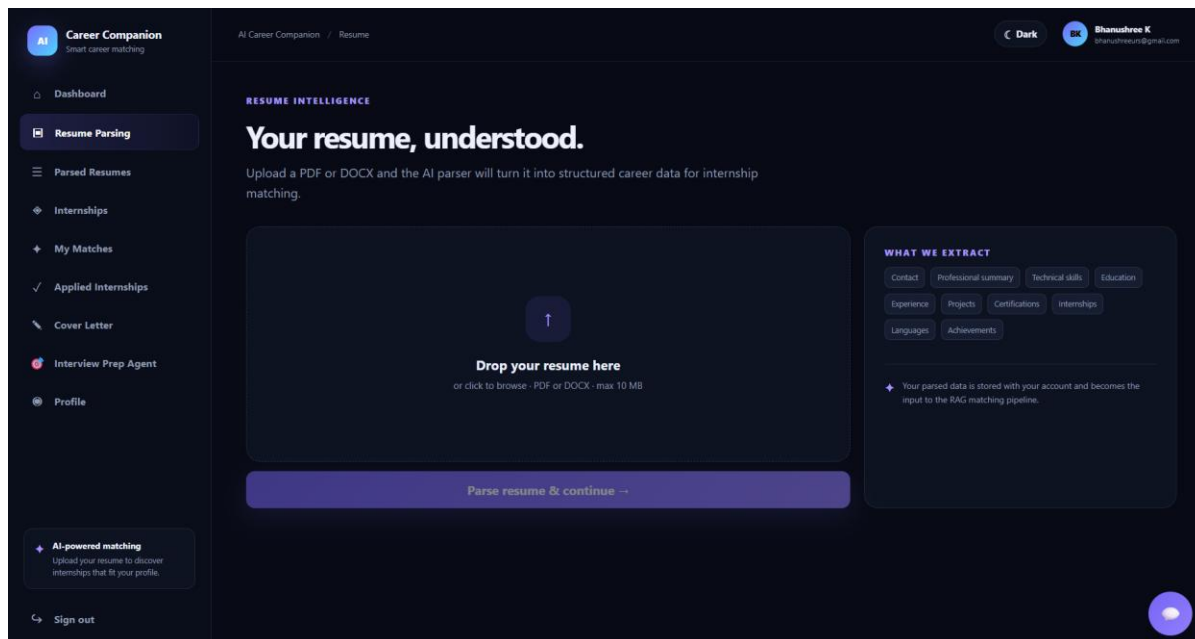


Figure 5. Resume Parsing page.

9.4 Internships

Feature: Users can browse the internship catalog and search/filter available opportunities.

How it works: The application exposes internship records through the backend and provides frontend controls for browsing and filtering. The catalog forms the source pool used by the semantic matching component.

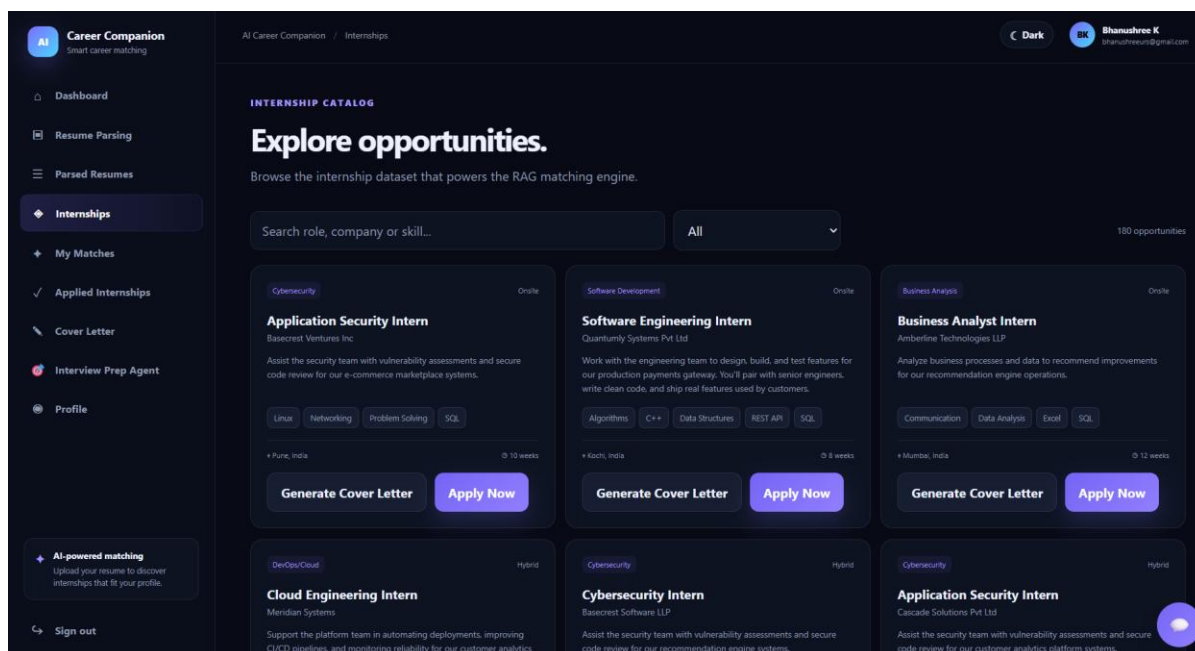


Figure 6. Internship catalog.

9.5 My Matches

Feature: Users can explicitly choose how many personalized internship matches they want and start the matching process.

How it works: Internship postings are represented using all-MiniLM-L6-v2 embeddings and indexed in FAISS. The latest parsed resume is converted into a semantic query, FAISS retrieves a candidate pool, and the candidates are re-ranked using skills, semantic similarity, education and location. The current UI provides 3, 5 and 10 match choices, defaults to 3, and starts matching only after the user clicks Find Matches.

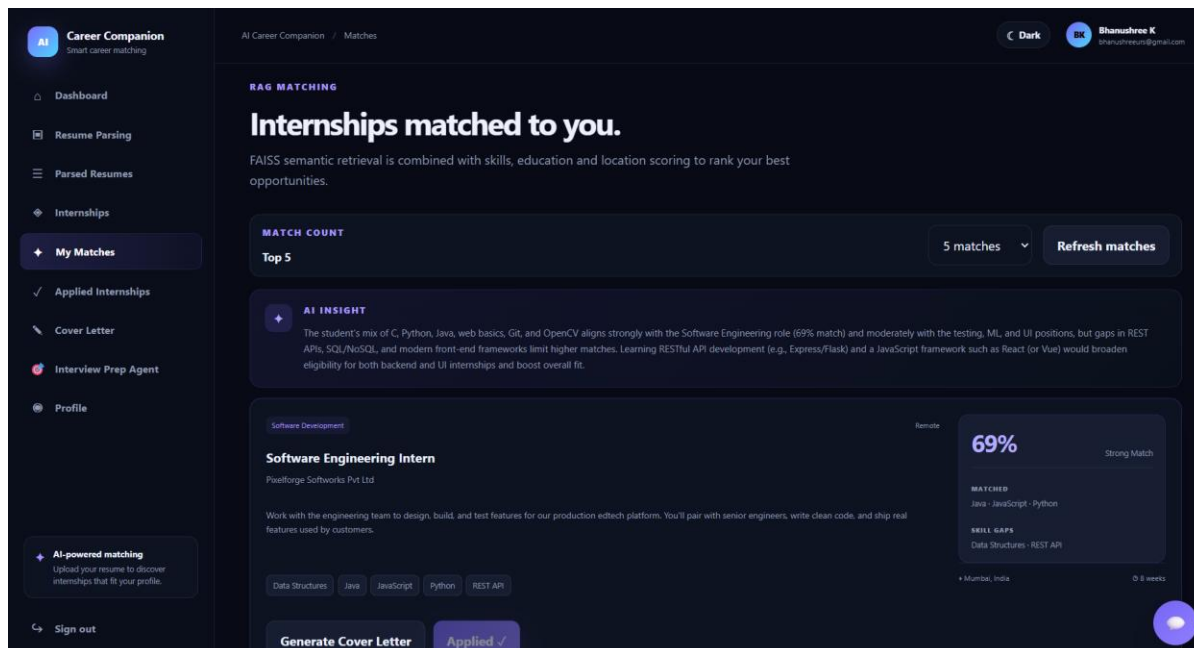


Figure 7. My Matches — user-selected internship matching.

9.6 Cover Letter

Feature: Users can generate an internship-specific cover letter.

How it works: The backend loads the selected internship and the user's parsed resume and sends those supplied facts to Groq for generation. The prompt instructs the model not to invent candidate skills, experience, dates, achievements or company facts. Generated cover letters are stored in PostgreSQL.

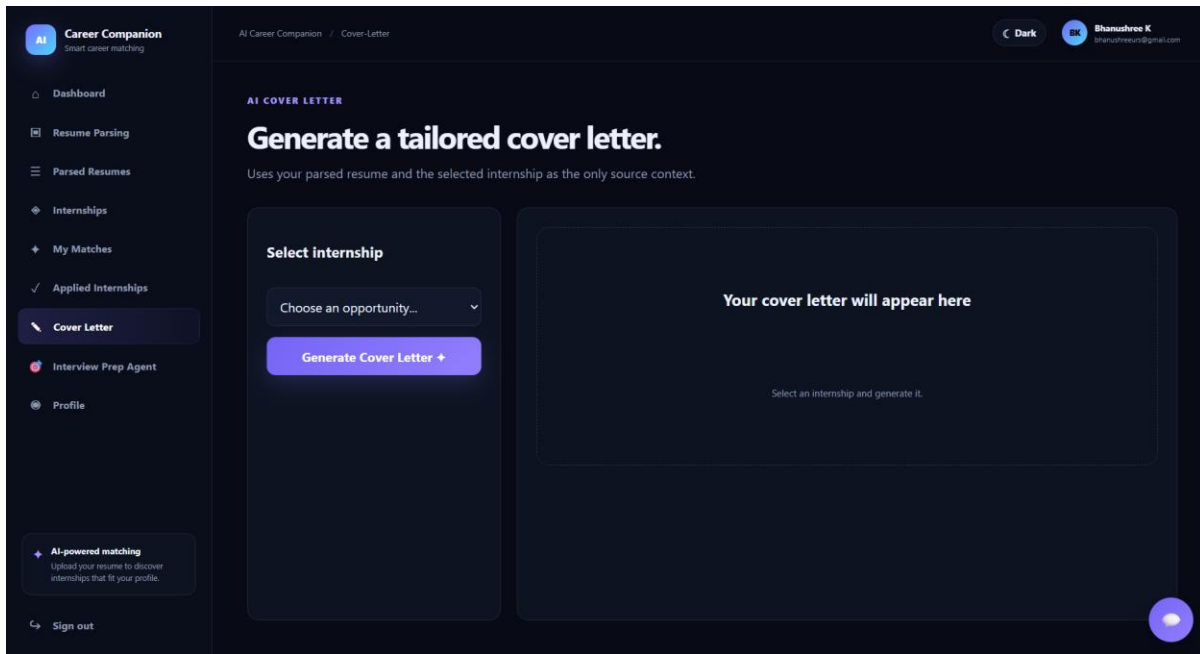


Figure 8. Cover Letter generation.

9.7 AI Product Assistant

Feature: The AI Product Assistant is a core internship-support feature developed during the internship. It helps users understand and navigate the Career Companion application through an AI assistant grounded in the project's product knowledge.

How it works: The Product Assistant retrieves relevant sections from the product-knowledge document through a dedicated FAISS index and sends the retrieved context together with recent chat history to Groq for response generation. It is a separate service from the Interview Preparation Agent.

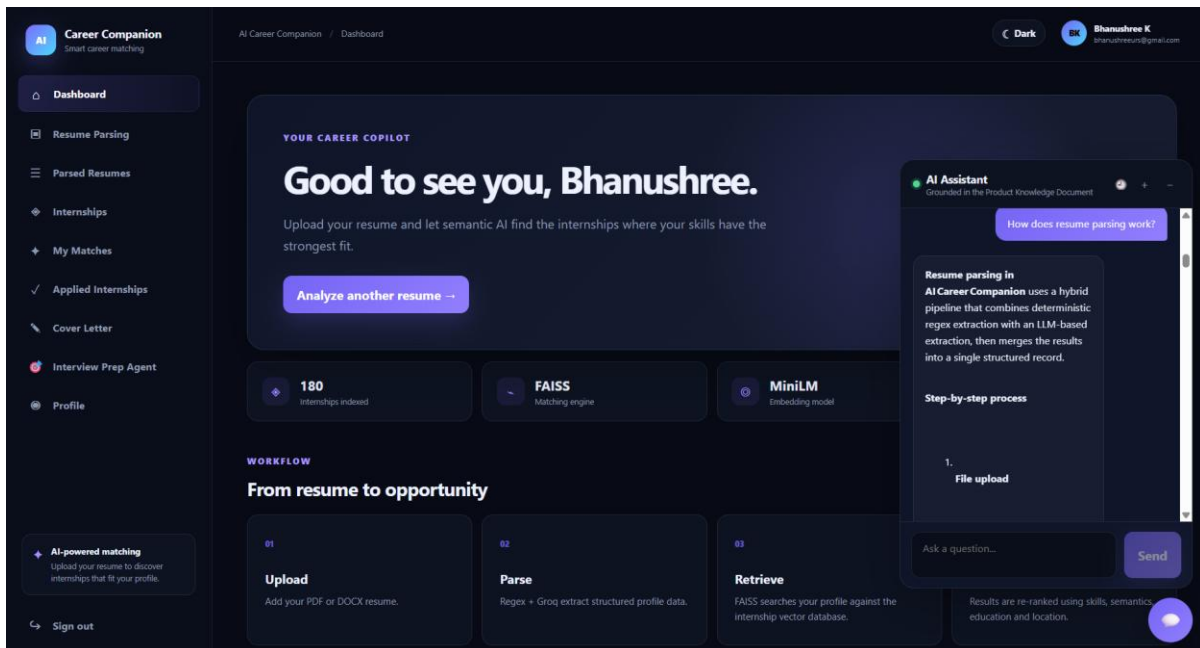


Figure 9. AI Product Assistant.

9.8 Interview Preparation Agent

Feature: A dedicated AI Interview Preparation Agent supports technical, HR/behavioral, company-specific and document-based interview preparation.

How it works: This agent is separate from the floating Product Assistant. It reuses the latest parsed resume, can use uploaded PDF/DOCX context, and retains the current interview-session conversation history. The combined context is sent to the configured Groq LLM for questions, explanations, model answers and study guidance.

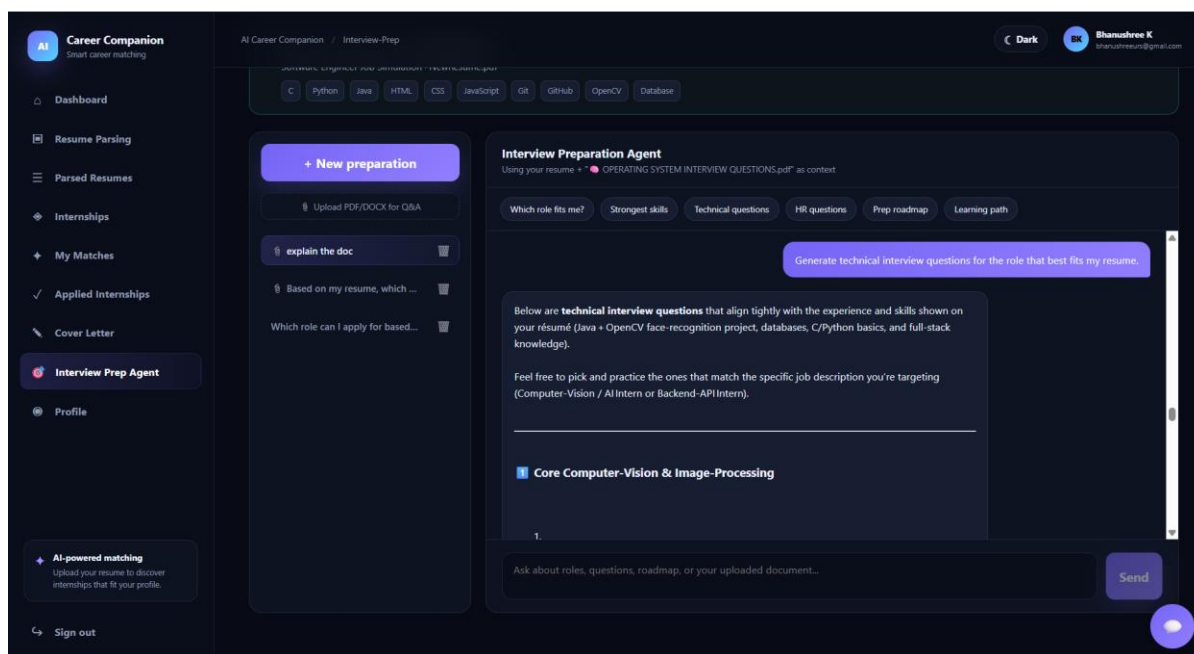


Figure 10. AI Interview Preparation.

9.9 Profile

Feature: Users can maintain a separate Career Profile.

How it works: The Career Profile is a manually managed persistent record and is intentionally separate from parsed-resume data. This allows the user to edit profile information without silently changing the stored parsed-resume record used for matching.

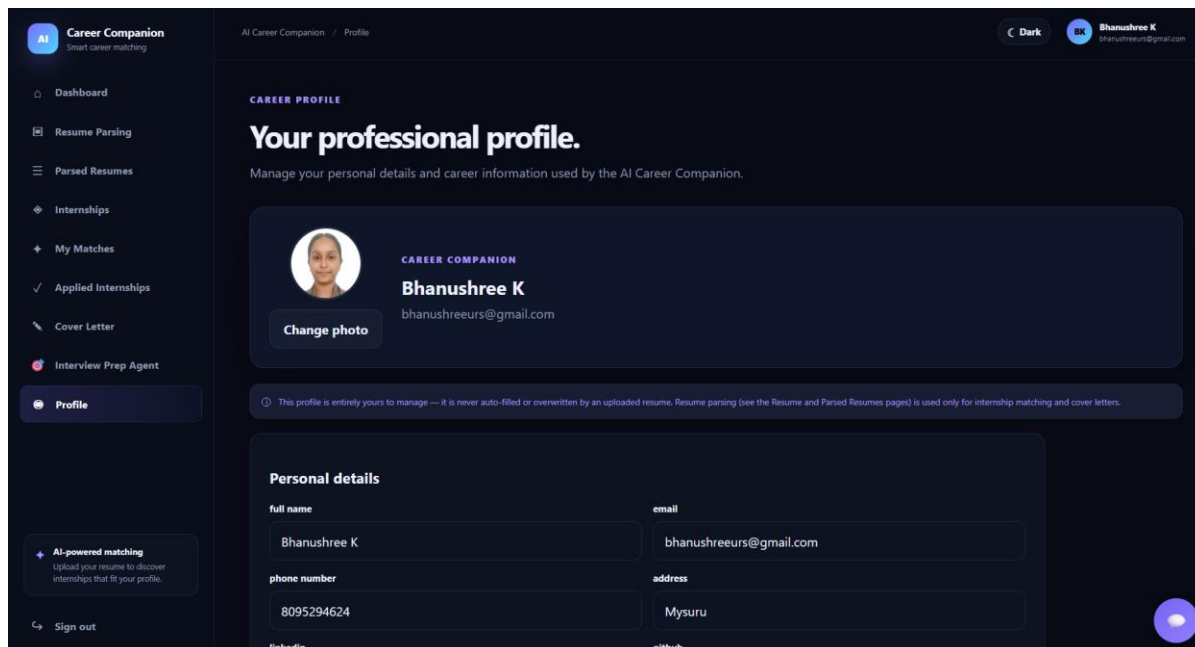


Figure 11. Career Profile.

9.10 Additional Features

The following supporting features are presented after the core post-login navigation and core internship/AI feature set: Parsed Resumes, Applied Internships, and Light / Dark Theme.

Parsed Resumes

Users can retain multiple uploaded/parsed resumes, review earlier records and reuse or re-run parsing on a selected resume.

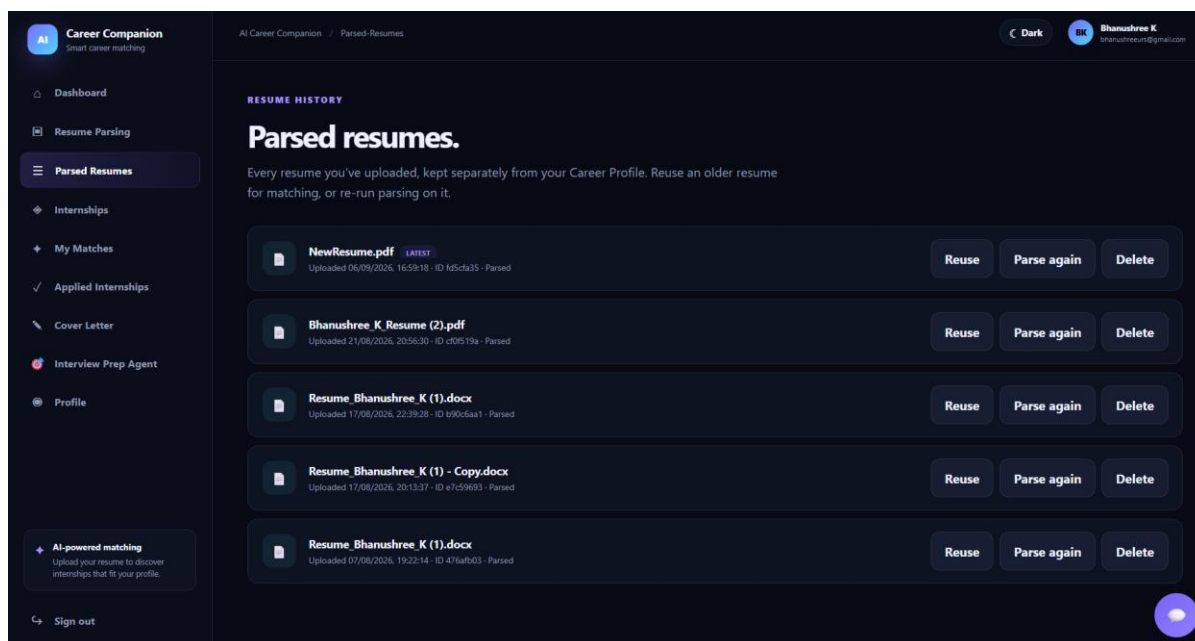


Figure 12. Parsed Resumes.

Applied Internships

Users can view internships they have applied to. Applications are persisted and duplicate applications for the same user and internship are prevented by the application's data constraints.

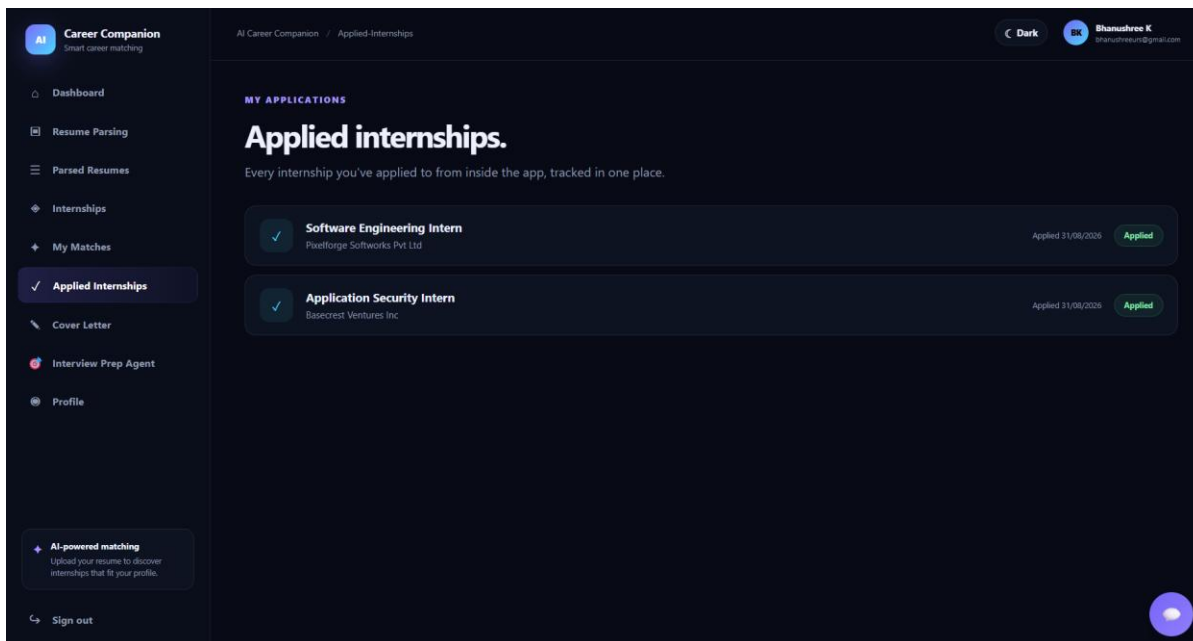


Figure 13. Applied Internships.

Light / Dark Theme

The frontend provides theme support for the application interface.

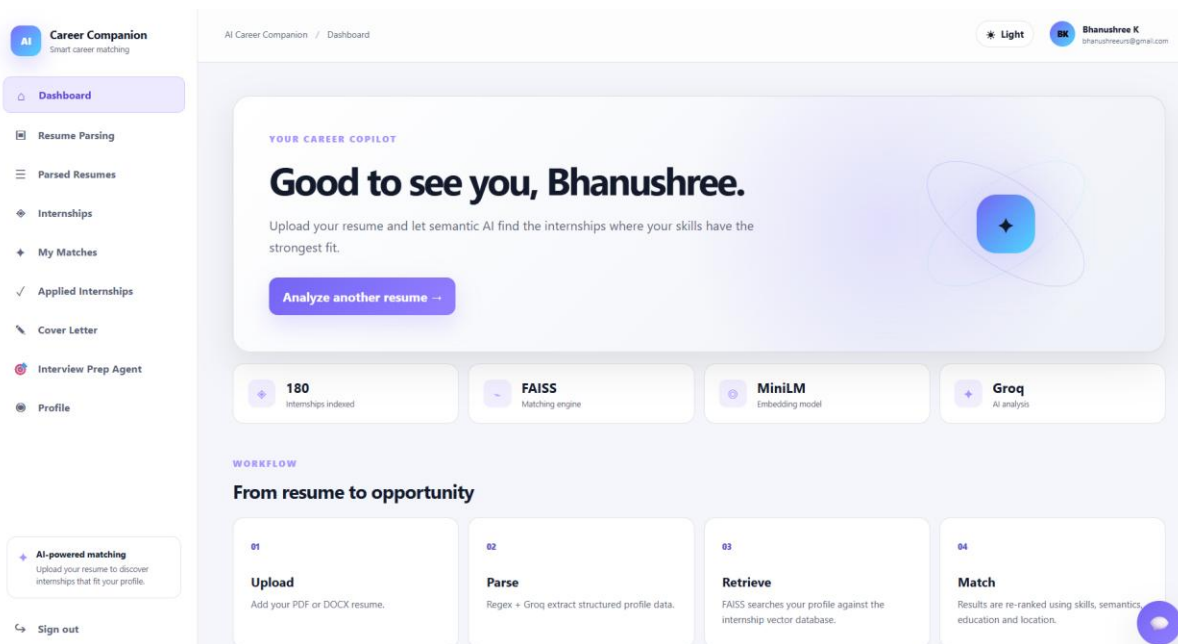


Figure 14. Light theme.

10. Testing

Testing in the current project focuses on functional behavior of the frontend-to-backend workflow, API validation, data ownership, and AI-assisted flows. The table lists final-version test cases supported by the implemented application. The two recently changed flows were verified in the current working build by running the project.

Test ID	Test Case	Expected Result	Actual / Implementation Result	Status
T001	Register with valid details	Account is created and user is taken to Login	Registration flow navigates to /login	Pass
T002	Login with valid credentials	JWT is issued and user reaches Dashboard	Login flow routes to Dashboard	Pass
T003	Upload valid PDF/DOCX resume	Resume is accepted and parsed	Hybrid parsing endpoint implemented	Pass
T004	Upload unsupported file type	Clear validation error is shown	Backend rejects unsupported extensions	Pass
T005	Open My Matches without clicking search	No matching request starts automatically	Matching is triggered only by Find Matches	Pass
T006	Select 3 / 5 / 10 matches	Selected count is sent to matching endpoint	Frontend passes k to the matching endpoint	Pass
T007	Apply to an internship twice	Duplicate application is prevented	Database unique constraint protects the pair	Pass
T008	Generate cover letter	Letter uses parsed resume + selected internship context	Groq-generated letter is saved	Pass
T009	Use Product Assistant	Relevant product-knowledge context is retrieved	Separate FAISS product index + Groq response	Pass
T010	Use Interview Preparation Agent	Resume/document/session context is available	Dedicated interview session and message flow implemented	Pass

Testing Types

- Functional Testing — registration, login, resume upload, matching, applications and feature workflows.
- API Testing — FastAPI endpoints and validation/error responses.
- UI Testing — navigation, forms, match selection and rendered AI content.
- Integration Testing — frontend communication with FastAPI and persistence in PostgreSQL.

- **AI / Retrieval Testing** — resume extraction, internship retrieval/re-ranking, Product Assistant retrieval and LLM generation.

11. Challenges & Solutions

The challenges below are based on implementation decisions visible in the supplied repository and the final workflow. They are presented as engineering challenges rather than generic AI-project statements.

Challenge	Solution
Resume extraction needs both precision and semantic understanding	Used a hybrid parser: Regex handles deterministic contact information while Groq/LLM handles contextual fields such as skills, education, projects and experience. A merge layer combines the outputs.
Semantic similarity alone is not enough for internship ranking	Retrieved a larger candidate pool from FAISS and re-ranked candidates using weighted skill, semantic, education and location scores. The result exposes matched skills and skill gaps.
Product Assistant and internship matching require different knowledge indexes	Kept separate FAISS indexes/directories for internship postings and product knowledge, while reusing the same cached MiniLM embedding model.
Interview preparation needs resume and document context without duplicating resume parsing	The Interview Preparation Agent reuses the existing parsed-resume record and existing PDF/DOCX extraction utilities, adding session history and optional uploaded document context.
Career Profile data should not silently change when a resume changes	Implemented Career Profile as a separate user-managed record. Parsed resumes remain a distinct concept used for matching and cover letters.
AI-generated content must avoid unsupported candidate facts	The cover-letter prompt limits generation to supplied candidate and internship information and instructs the model not to invent facts.
Matching should not start before the user chooses the desired result count	Changed My Matches so navigation does not automatically call the matching endpoint. The user selects 3, 5 or 10 and explicitly clicks Find Matches.

12. Final Project Status

12.1 Completed Features

- User registration, login, logout, JWT authentication and protected routes.
- Password reset/change flow.
- PDF/DOCX resume upload and hybrid Regex + Groq LLM parsing.
- Parsed resume storage, history, reuse and re-parsing.
- Internship catalog with search and domain filtering.
- FAISS + all-MiniLM-L6-v2 internship matching with composite re-ranking.
- Match explanations including percentage, label, matched skills and skill gaps.
- Career Profile management.
- Applied Internships tracking with duplicate-application protection.
- AI-generated internship-specific cover letters.
- AI Product Assistant with product-knowledge RAG.
- Separate AI Interview Preparation Agent with resume/document/session context.
- Light/dark theme support and integrated application navigation.

12.2 Final Submission Status

- Project implementation completed.
- Final documentation prepared.
- Final project demonstration prepared.

12.3 Final Outcome

The project implements an integrated AI career companion that takes a candidate from authentication and resume parsing through internship discovery, personalized matching, application support, the AI Product Assistant and interview preparation. The application is organized as a React/Vite frontend backed by FastAPI and PostgreSQL, with FAISS-based vector retrieval and Groq-powered AI services.

The current working version includes the requested interaction changes for the final flow: a newly registered user is returned to the Login page instead of being automatically logged in, and My Matches waits for the user to select 3, 5 or 10 matches and explicitly start the search. The AI Product Assistant is documented as a core internship feature and precedes the subsequently developed Interview Preparation Agent.

Project duration and official sprint dates were not supplied with the source files used for this document and therefore remain explicit confirmation fields rather than guessed values.

12.4 Conclusion

The project demonstrates the integration of web application development, secure user management, document processing, semantic retrieval, vector search, LLM-based generation, and AI-assisted career workflows in a single internship-oriented application.

Prepared by: Bhanushree K | Mentor: Vijay Parmar